# Announcements

# Topic 7: Binary Search Trees

By: Professor Lynam

# General Trees

- Definition: *General Tree*
- *A general tree T is empty or is a finite set of Nodes N such that there is a specially-designated root node r* ∈ N with the remaining N – r nodes partitioned into $m \geq 0$ *disjoint subtrees of T*
- Binary Search Trees ⊆ Binary Trees ⊆ General Trees
- Note that this is a recursive definition, meaning you can have sub-trees, and trees being forming by attaching existing trees
- A node in a general tree may have any non-negative quantity of "children"

# Binary Trees

- Definition: *Binary Tree*
- *A binary tree is a general tree such that* $0 \leq m \leq 2$.

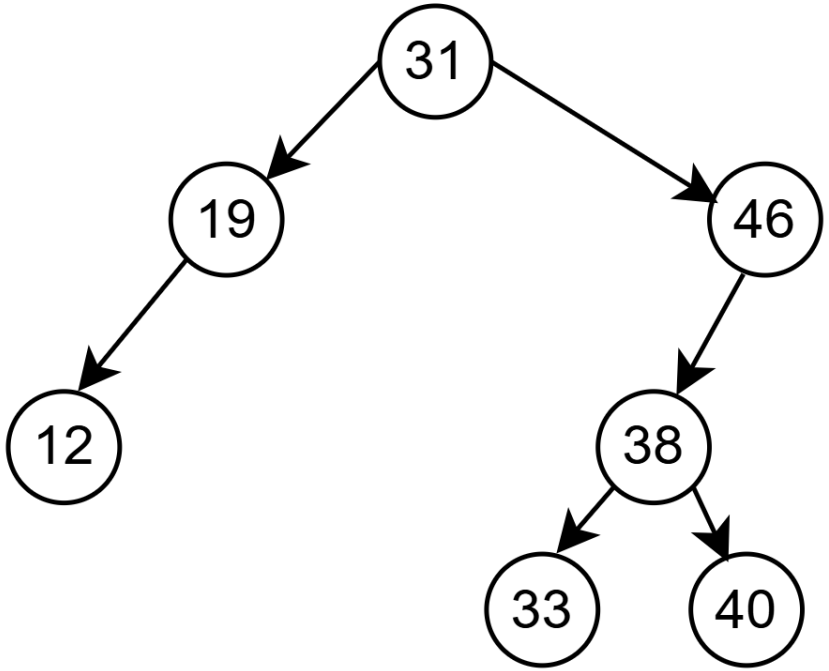
- We call the node “higher” in the tree the parent node, and assuming it has two children, it has a “left” and “right” child node
- Example applications of binary trees include:
- Binary Space Partitioning (BSP) in video games
- Huffman Coding (lossless data compression)
- Heaps (priority queues)
- Expression Trees (intermediate expression representation)
- And last, but not least…

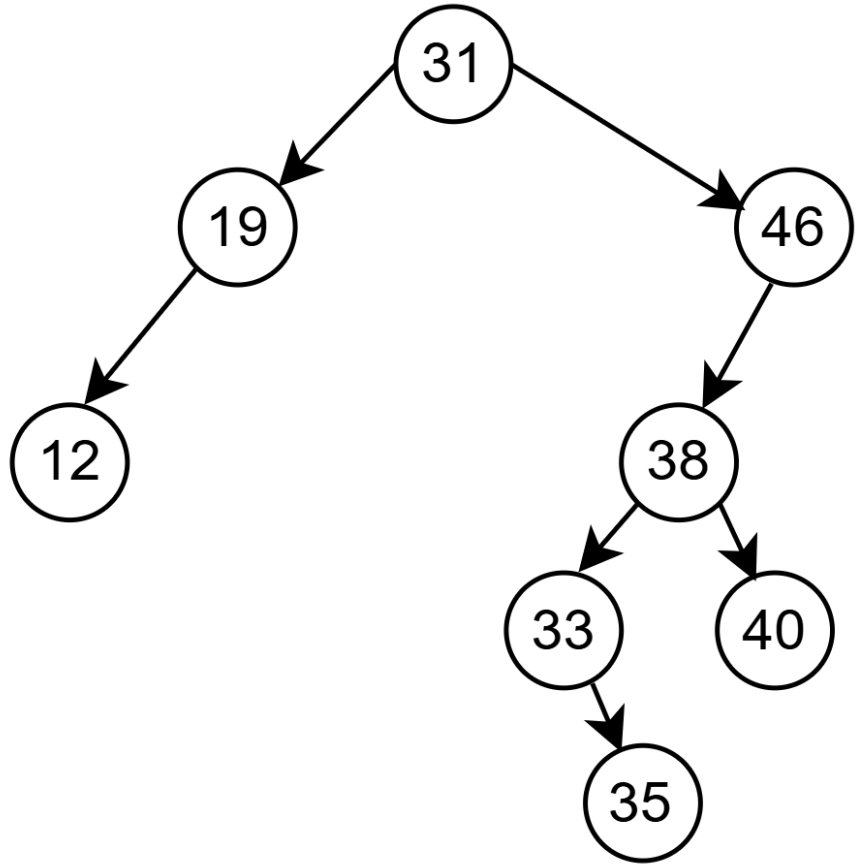
# Binary Search Trees (BSTs)

- Definition: *Binary Search Tree*
- *A binary search tree (BST) is a binary tree in which any key value held in a node is greater than any key in that node's left subtree and is less than any key in its right subtree.*

- What about equal key values, you might ask?
- We can extend the definition to store ties on one side or the other (but not both!) of existing matches. (However, in many BST applications, data values are unique.)

# BST Insertions




- Let's insert 35 into this BST. What are the steps?
- Just search for 35, and insert into the fringe where the search fails